applied sciences

MDPI

Article

AgentReport: A Multi-Agent LLM Approach for Automated and Reproducible Bug Report Generation

Seojin Choi [1] and Geunseok Yang [2,*]

[1] Department of Computer Applied Mathematics, Hankyong National University, Anseong 17579, Republic of Korea; insui12@hknu.ac.kr
[2] Department of Computer Applied Mathematics, Computer System Institute, Hankyong National University, Anseong 17579, Republic of Korea
* Correspondence: gsyang@hknu.ac.kr

Abstract

Bug reports in open-source projects are often incomplete or low in quality, which reduces maintenance efficiency. To address this issue, we propose AgentReport, a multi-agent pipeline based on large language models (LLMs). AgentReport integrates QLoRA-4bit lightweight fine-tuning, CTQRS (Completeness, Traceability, Quantifiability, Reproducibility, Specificity) structured prompting, Chain-of-Thought reasoning, and one-shot exemplar within seven modules: Data, Prompt, Fine-tuning, Generation, Evaluation, Reporting, and Controller. Using 3966 summary–report pairs from Bugzilla, AgentReport achieved 80.5% in CTQRS, 84.6% in ROUGE-1 Recall, 56.8% in ROUGE-1 F1, and 86.4% in Sentence-BERT (SBERT). Compared with the baseline (77.0% CTQRS, 61.0% ROUGE-1 Recall, 85.0% SBERT), AgentReport improved CTQRS by 3.5 percentage points, Recall by 23.6 points, and SBERT by 1.4 points. The inclusion of F1 complemented Recall-only evaluation, offering a balanced framework that covers structural completeness (CTQRS), lexical coverage and precision (ROUGE-1 Recall/F1), and semantic consistency (SBERT). This modular design enables consistent experimentation and flexible scaling, providing practical evidence that multi-agent LLM pipelines can generate higher-quality bug reports for software maintenance.

Keywords: bug report automation; software maintenance; large language models; multi-agent systems; Quantized LoRA; chain-of-thought reasoning; CTQRS; ROUGE; Sentence-BERT; one-shot learning

Academic Editor: Douglas O'Shaughnessy

Received: 30 September 2025
Revised: 2 November 2025
Accepted: 6 November 2025
Published: 10 November 2025

Citation: Choi, S.; Yang, G. AgentReport: A Multi-Agent LLM Approach for Automated and Reproducible Bug Report Generation. *Appl. Sci.* **2025**, *15*, 11931. https://doi.org/10.3390/app152211931

1. Introduction

Bug reports are essential artifacts for identifying and tracking defects during software development and maintenance. However, in open-source projects, reports are often written by non-expert contributors and tend to be incomplete or recorded in free-form styles. Such reports complicate the processes of reproducing and fixing defects, leading to lower software quality and reduced productivity.

Although this study primarily focuses on open-source projects using Bugzilla [1], similar challenges of incomplete or inconsistent issue reports have been observed in commercial, closed-source platforms such as Jira [1]. These parallels indicate that the problem of low-quality issue reports is not confined to open-source ecosystems but represents a broader challenge in software maintenance practices.

Empirical studies have quantified the maintenance overhead caused by poor-quality bug reports.

Bettenburg et al. [2] found that developers spend up to 45% of their bug-fixing time clarifying unclear or incomplete reports, and Medeiros et al. [3] observed that low-quality crash reports delay issue resolution by an average of 3 days.

These statistics concretely illustrate how inconsistent or incomplete reports impose measurable time and cost burdens on software maintenance teams.

Consequently, the problem of automatically structuring unformatted reports and supplementing missing information has long been recognized as a significant research challenge in software engineering [2,4].

Recently, large language models (LLMs) such as GPT-3 and GPT-4 have shown the potential to automate software reports through few-shot learning and advanced reasoning capabilities [5–7]. LLMs, trained on massive datasets, have acquired strong contextual understanding and natural language processing abilities, and they can generate text under specific constraints through prompt design and fine-tuning techniques [8]. Nevertheless, prior studies have mostly relied on template-based approaches or simple LoRA fine-tuning [8], which do not adequately incorporate quality metrics such as CTQRS (Completeness, Traceability, Quantifiability, Reproducibility, Specificity) [9]. In addition, the effect of one-shot exemplar has not been systematically evaluated, and performance assessments have focused primarily on ROUGE-1 Recall [10], which fails to sufficiently balance precision and recall.

To address these limitations, this study proposes AgentReport, a multi-agent LLM pipeline. AgentReport integrates QLoRA-4bit lightweight fine-tuning [11], CTQRS-based structured prompts [9], Chain-of-Thought (CoT) reasoning [12], and one-shot exemplar provision [5]. The architecture consists of Data, Prompt, Fine-tuning, Generation, Evaluation, Reporting, and Controller modules, ensuring reproducibility, scalability, and modularity. This design also aligns with recent research on LLM-based autonomous agents [13] and memory mechanisms [14], which emphasize long-term applicability.

Unlike general multi-agent frameworks such as ReAct [15], AutoGen [16], and LangChain [17], which primarily focus on task delegation or conversational planning, AgentReport introduces a domain-specific coordination and evaluation architecture designed for automated bug report generation. Each agent is explicitly defined by its fixed responsibilities, input/output contracts, and integration with quantitative evaluation mechanisms (CTQRS, ROUGE, SBERT), enabling reproducibility and modular substitution without ad hoc orchestration.

Using 3966 summary–report pairs collected from Bugzilla, AgentReport consistently outperformed the Baseline, showing improvements of +3.5 percentage points in CTQRS, +23.6 percentage points in ROUGE-1 Recall, and +1.4 percentage points in SBERT. It also achieved 56.8% in the newly introduced ROUGE-1 F1 evaluation, mitigating recall bias. These results show that AgentReport can generate high-quality bug reports that combine structural completeness, lexical fidelity, and semantic consistency.

The main contributions of this work are summarized as follows:

- Proposal of a multi-agent modular pipeline: We designed an architecture composed of Data, Prompt, Fine-tuning, Generation, Evaluation, Reporting, and Controller modules. This modular design overcomes the limitations of single-pipeline approaches and provides a reproducible and extensible framework.
- Introduction of a new evaluation baseline: We introduced ROUGE-1 F1, which had not been reported in previous studies, and achieved 56.8%. This provides a new evaluation standard that complements recall-oriented assessments and incorporates precision.
- Validation of performance improvements over the Baseline: In experiments with 3966 Bugzilla summary–report pairs, AgentReport improved CTQRS (+3.5 pp), ROUGE-1 Recall (+23.6 pp), and SBERT (+1.4 pp). These results confirm that Agen-

tReport delivers overall enhancements in structural completeness, lexical fidelity, and semantic consistency.

This study proposes AgentReport, a modular multi-agent LLM pipeline designed for automated bug report generation. The framework integrates QLoRA-4bit fine-tuning, CTQRS-based structured prompting, CoT reasoning, and one-shot exemplar retrieval to enhance report completeness, lexical fidelity, and semantic consistency. The remainder of this paper is organized as follows. Section 2 reviews the background of bug report quality variation and summarizes prior research trends in automation. Section 3 presents the proposed AgentReport framework, explaining its multi-agent modular architecture and the specific responsibilities of each component. Section 4 describes the experimental settings, datasets, and evaluation metrics, followed by performance results and comparative analysis. Section 5 discusses the implications of the findings and examines the threats to validity. Section 6 reviews related work to position our contribution within the broader literature. Finally, Section 7 concludes the paper and highlights directions for future research.

2. Background Knowledge

2.1. Variability in Bug Report Quality and the Need for Structuring

Bug reports serve as essential artifacts for understanding and resolving defects during software maintenance. However, in real-world projects, the quality of these reports often varies significantly depending on the reporter's experience, writing habits, and tool usage. This problem is particularly pronounced in open-source environments where contributors come from diverse backgrounds. As a result, fundamental information such as reproduction steps, execution environment, and expected versus actual outcomes is often missing or ambiguously described. Such incomplete reports hinder defect reproduction, require additional verification during analysis, and ultimately lead to delayed fixes and increased maintenance costs [2].

Figure 1 illustrates a representative example of this variability. Report (a) is incomplete, lacking essential environment details and reproduction steps, while its description of expected behavior remains vague. In contrast, report (b) presents clear reproduction procedures, environment settings, and expected outcomes, which enable developers to reproduce and resolve the defect efficiently. This comparison highlights that the structural completeness of a report is directly tied to maintenance effectiveness.

Several metrics have been employed to quantitatively evaluate bug report quality. CTQRS measures the extent to which a report includes critical information required for defect resolution, providing a direct assessment of structural completeness [9]. ROUGE metrics calculate lexical overlap with reference reports to indicate whether key terms are adequately covered [10]. SBERT-based metrics leverage sentence embeddings to assess semantic consistency with reference reports [18]. Together, these metrics enable a multidimensional evaluation of structural fidelity, lexical coverage, and semantic alignment.

Nevertheless, quality evaluation alone does not fundamentally address challenges encountered in practice. While it is possible to identify reports with low quality scores, developers still face a manual burden unless these reports are automatically supplemented or restructured. This problem becomes particularly severe in large-scale projects where a continuous influx of reports makes it difficult to manage variability through diagnostic approaches alone. To overcome this limitation, recent research has expanded beyond evaluation toward automatic supplementation and structuring of incomplete reports using natural language processing and LLMs. This direction holds promise for reducing variability in report quality, accelerating defect resolution, and ultimately improving maintenance productivity.

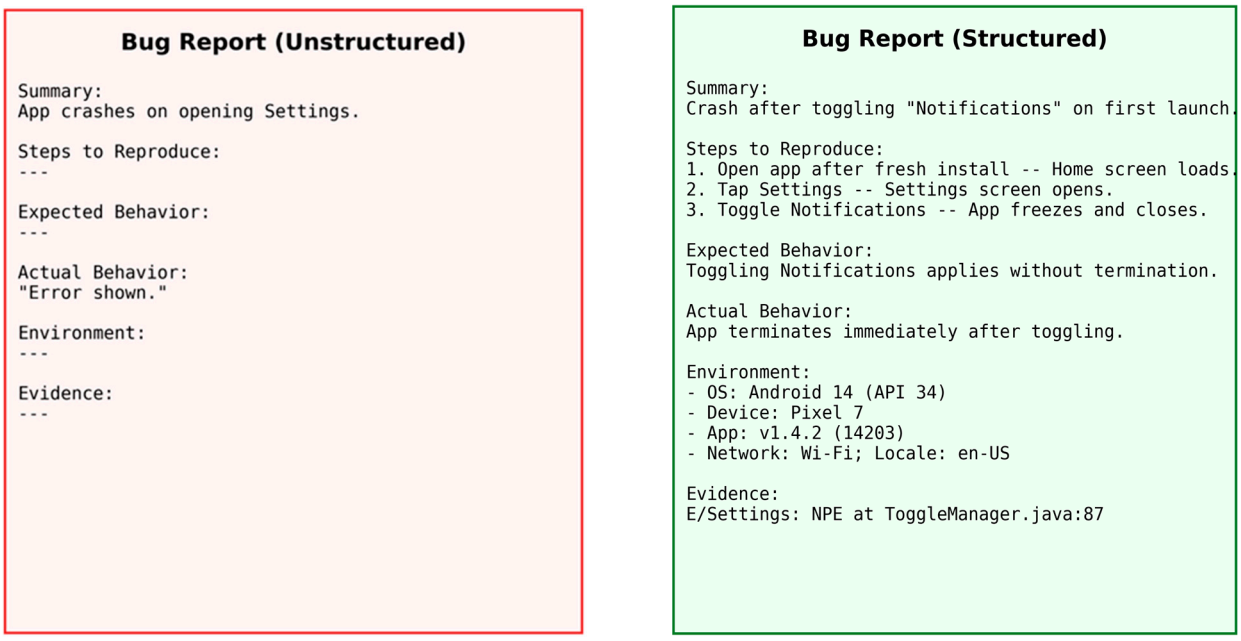

Figure 1. Examples of bug reports: (**a**) Incomplete bug report missing essential details such as reproduction steps, environment information, and expected results. (**b**) Structured bug report with clearly specified reproduction steps, environment configuration, and expected results.

2.2. Trends in Automated Bug Report Research

A variety of automation techniques have been proposed to address the issue of bug report quality. Early approaches often relied on template-based methods. Well-known bug tracking systems such as Bugzilla required reporters to complete predefined forms, which partially alleviated problems such as the omission of essential fields including reproduction steps and expected results. While this method helped reduce variation in quality, it also presented drawbacks: it was difficult to flexibly reflect project-specific requirements, and it limited the autonomy of reporters. Reports therefore often achieved formal consistency but lacked the contextual richness needed for effective maintenance.

Later research explored the use of machine learning (ML) techniques for automation. For example, text classification models were applied to detect duplicate bug reports [19,20] or to predict their quality level [21,22]. These methods provided the advantage of learning structural features in a data-driven manner and automatically identifying report quality. However, performance degraded significantly when sufficient training data were unavailable. In particular, the lack of domain-specific datasets reduced generalizability, meaning that models often performed well on one project but could not be easily transferred to others. Furthermore, because these approaches focused primarily on classification or prediction, they did not provide mechanisms for structuring reports or automatically supplementing missing information.

More recently, the rise in LLMs has brought a new turning point in research on bug report automation. Advanced LLMs, such as those in the GPT family, have substantially improved natural language understanding and generation capabilities through large-scale text training [5–7]. With prompt engineering and fine-tuning, they can now be adapted to specialized tasks such as software report generation. Several studies have experimented with LLMs to summarize incomplete reports or to automatically supplement missing fields [23–26], and these efforts have shown greater potential than template- or ML-based approaches. Nevertheless, most of these studies have relied on a single model and a single pipeline. As a result, it has been difficult to ensure consistent outputs for identical

inputs, and incorporating new evaluation metrics or strategies often required redesigning the entire pipeline. Limitations therefore remain in terms of reproducibility, modularity, and scalability.

Research on bug report automation has thus evolved from template-based approaches to ML-driven classification and prediction, and more recently to LLM-based generation and supplementation. Despite these advances, the challenge of reliably generating high-quality reports that remain consistent and scalable across diverse environments has not yet been fully resolved. This gap in the literature underscores the need for new approaches such as agent-based modular pipelines and provides a strong rationale for the development of the proposed AgentReport framework.

2.3. Baseline Studies and Their Limitations

The advancement of LLMs has marked a significant turning point in research on automated bug report generation. Through training on vast amounts of data, LLMs have acquired strong contextual understanding and natural language generation capabilities [5–7]. With prompt engineering and fine-tuning techniques, LLMs can be guided to produce reports that reflect structural constraints [8,11]. These characteristics open the possibility of directly addressing challenges such as supplementing unstructured reports and enforcing structured formats, which earlier template-based or machine learning methods could not adequately resolve.

However, applying LLMs within a single pipeline poses several limitations. First, outputs are often difficult to reproduce when experimental settings or input conditions change. Second, the reliance on a single, monolithic model structure hinders the flexible integration of new features or techniques. Moreover, when faced with incomplete reports, LLMs may misinterpret context or generate outputs lacking consistency across sections. While this LLM-only approach shows potential performance improvements, it fails to ensure the structural stability necessary to support long-term research and industrial adoption.

As an attempt to address these shortcomings, Acharya and Ginde [1] proposed a Retrieval-Augmented Generation (RAG)–based method for bug report automation. By retrieving external context and providing it to the LLM, this approach showed improvements in report quality. Nevertheless, RAG remains dependent on a single model and a single pipeline, leaving fundamental issues of modularity, reproducibility, and scalability unresolved.

Therefore, advancing bug report automation requires moving beyond simple LLM application toward an agent-based modular pipeline. In such an approach, tasks such as data processing, prompt design, fine-tuning, report generation, and evaluation are separated into independent modules. For instance, a structure consisting of a Data Agent, Prompt Agent, Fine-tuning Agent, Generation Agent, and Evaluation Agent enables the replacement or expansion of individual modules without redesigning the entire system when introducing new evaluation metrics or prompt strategies. This modularity supports repeated validation in academic research and provides long-term maintainability and adaptability in industrial environments.

To embody this direction, the present study proposes AgentReport. The proposed method not only aims at short-term performance improvements but also provides a structural alternative that ensures long-term stability and scalability, thereby opening new possibilities for both research on bug report automation and its practical applications.

Existing multi-agent frameworks such as ReAct [15], AutoGen [16], and LangChain [17] have demonstrated the general feasibility of agent coordination and tool usage in reasoning or dialogue tasks. However, these systems are primarily dynamic and stochastic, which makes them unsuitable for domains that require strict reproducibility and deterministic

evaluation. Their coordination mechanisms depend on open-ended message exchanges among agents, leading to variations in output even under identical conditions. In contrast, AgentReport formalizes a static coordination flow in which seven agents (Data, Prompt, Fine-tuning, Generation, Evaluation, Reporting, and Controller) operate sequentially with fixed responsibilities. This design sacrifices conversational flexibility but ensures consistent and reproducible outcomes essential for scientific validation. Furthermore, AgentReport integrates a domain-specific Evaluation Agent that automatically measures CTQRS, ROUGE, and SBERT metrics, providing a structured quantitative evaluation loop that general frameworks do not support.

Throughout this paper, the term *Agent* denotes modular components that carry out specific functions within a coordinated framework rather than fully autonomous entities. The architecture is inspired by the principles of multi-agent systems, yet it prioritizes reproducibility, controllability, and scalability over independent reasoning or communication between agents.

3. Methodology

The proposed AgentReport pipeline integrates the entire workflow in an agent-based modular architecture, spanning data preprocessing, prompt assembly, model training, report generation, quality evaluation, and results consolidation. Each module, referred to as an Agent, performs well-defined tasks (e.g., Data, Prompt, Finetuning, Generation) under the orchestration of the Controller Agent. These Agents are not autonomous actors but controlled components designed to ensure deterministic execution and reproducibility across experiments. Figure 2 illustrates the end-to-end flow, and the responsibilities of each agent are summarized as follows. The Data Agent cleans the source dataset and partitions it into training, validation, and test sets so that downstream modules can operate reliably. The Prompt Agent constructs the model input by combining CTQRS-based instructions, chain-of-thought reasoning, and retrieval of a one-shot example. The Finetuning Agent adapts a pretrained language model with QLoRA 4-bit fine-tuning to reflect these prompts. The Generation Agent produces bug reports using the trained adapter parameters together with the constructed prompts. The Evaluation Agent verifies quality using CTQRS, ROUGE, and SBERT metrics. The Reporting Agent aggregates the evaluation results and organizes them in a comparable format. The Controller Agent oversees the entire execution and integrates the stages from data preparation through performance analysis into a single coherent process.

3.1. Data Preprocessing and Data Agent

This study utilized a Bugzilla-based bug report dataset that had been established in prior work [27]. In the initial stage, approximately 15,000 reports were collected, from which only those marked as “fixed” or “closed” were selected to construct the candidate training dataset. Resolved bug reports generally contain essential information such as reproduction steps, execution environment, expected results, and actual results, making them more suitable for providing structured cues for model training compared to unresolved reports.

However, the raw data still exhibited issues of inconsistency and missing information. Reports containing only stack traces, partial code fragments, or lacking any mention of environmental details or expected outcomes were considered detrimental to training quality and were therefore removed. To address this, a two-stage refinement procedure was implemented.

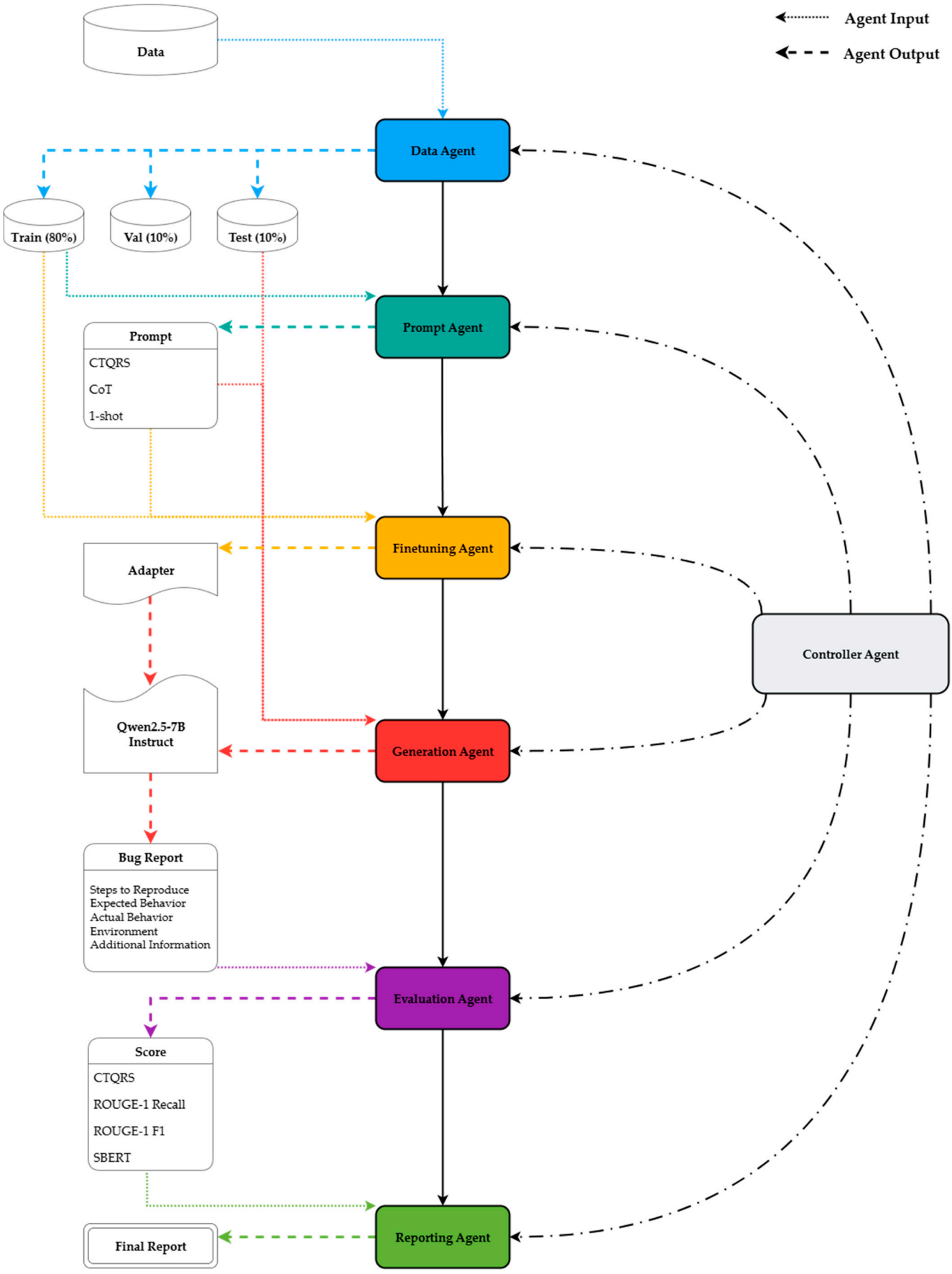

Figure 2. AgentReport Pipeline Architecture (Overall Agent Workflow).

In the first stage, regular-expression-based filtering was applied, retaining only those reports that included all of the following components: summary, steps to reproduce, expected result, actual result, and additional information. In the second stage, automated evaluation using the CTQRS metric was conducted, and only reports scoring 14 or higher were retained [9]. This threshold was adopted from prior research as a criterion for identifying high-quality reports, serving as a mechanism to ensure structural completeness and informational adequacy.

Through this process, a final set of 3966 reports was selected, with an additional manual review of 200 samples conducted by the researchers to further validate appropriateness. The final dataset can also be accessed through the GitHub repository released by the original authors [27] (commit 8ba64c4, accessed on 30 November 2025), ensuring fairness and reproducibility in data usage.

The dataset was partitioned into training, validation, and test sets at an 8:1:1 ratio, with a fixed random seed to guarantee consistency across repeated splits. This approach prevents data leakage and ensures fair conditions when comparing different strategies.

Subsequent preprocessing and partitioning were automatically managed by the Data Agent. As the first module in the overall pipeline, the Data Agent takes the finalized dataset, divides it into training, validation, and test sets, and converts each report pair into a standardized input format, such as [Input Summary] and [Reference Report]. This process ensures that data are delivered in a consistent format to subsequent modules, providing a stable foundation for the Prompt Agent. Additionally, the Data Agent supports reusing existing splits or performing new partitions with fixed random seeds when needed, thereby guaranteeing reproducibility and fairness across diverse experimental settings.

3.2. Prompt Agent

Figure 3 illustrates an example of a prompt assembled by the Prompt Agent. This prompt is designed to include CTQRS-based structured instructions, step-wise self-check guidance (CoT), and a one-shot exemplar retrieved from the training dataset. The prompt converts an input summary into a high-quality bug report.

The Prompt Agent takes as input the partitioned dataset provided by the Data Agent. Instead of allowing the model to respond in free form, it guides the model to strictly follow the CTQRS framework, which includes the following seven sections: Summary, Steps to Reproduce, Expected Result, Actual Result, Environment Information, Evidence, and Additional Information. The prompt enforces the inclusion of all seven items, thereby reducing the likelihood of incomplete or ambiguous reports and ensuring that essential information for defect reproduction is reliably captured.

The prompt also incorporates a CoT instruction. This directs the model not to produce its output all at once, but to perform a step-wise self-check to detect omissions or inconsistencies. Through this process, the model can revise its output when necessary, enhancing the logical consistency and completeness of the report.

In addition, the Prompt Agent performs FAISS-based retrieval of one-shot exemplar. The input summary is converted into BAAI/bge-large-en-v1.5 embeddings [28], which are then used to search for the most similar case within the training set. The retrieved example is inserted into the prompt as [Example Input]/[Example Output] blocks. This allows the model to generate its output not only under structural constraints but also with reference to real examples, while restricting retrieval to the training data ensures that no data leakage occurs.

```
"You are an expert QA engineer. Convert the user's unstructured
input into a CTQRS-maximized bug report.

[Additional Info #1: CTQRS-structured guidance]
Guide the model to strictly follow CTQRS criteria (required sections, numbered procedure, inclusion of environment and evidence, etc.).

[Additional Info #2: CoT (Self-check) instruction]
Before producing the output, have the model perform step-wise self-checks to address omissions and inconsistencies.

[Additional Info #3: FAISS-based one-shot exemplar]
Retrieve one similar example from the training data and insert it as [Example Input]/[Example Output] blocks.

--- Relevant Examples ---
[Example Input]
<Example Summary>

[Example Output]
<Generated Bug Report>
--- End of Examples ---

[Input to Convert]
<summary>

[Output Bug Report]
"
```

Figure 3. Example Prompt Assembled by the Prompt Agent.

The prompt is assembled in a unified format containing the markers "[Input to Convert] <summary>" and "[Output Bug Report]", and is then passed to the Fine-tuning Agent and Generation Agent. This design integrates structural constraints, self-checking, and example-based guidance, thereby improving the consistency, completeness, and semantic fidelity of the model outputs. Since all processes are executed under fixed conditions and standardized rules, the same prompt can always be reproduced, ensuring fairness and repeatability in experiments.

The Prompt Agent therefore plays a central role in controlling the format and improving the quality of outputs, rather than merely passing data between modules. CTQRS instructions guarantee structural completeness, the CoT directive reduces omissions and inconsistencies through self-review, and the one-shot exemplar provides contextual grounding for realistic outputs. Together, these features enable the model to move beyond simple language generation and produce high-quality bug reports that are practical for software maintenance.

3.3. Fine-Tuning Agent

The Fine-tuning Agent takes the structured prompt assembled by the Prompt Agent as input and fine-tunes the pretrained language model so that CTQRS instructions, step-wise reasoning (CoT), and the one-shot exemplar strategy are effectively reflected during the

generation process. This module goes beyond simply relaying input; it plays a central role in internalizing structural constraints into the model during training.

In this study, we applied the QLoRA-4bit method [11] to the Qwen2.5-7B-Instruct model [29]. QLoRA combines quantization with the LoRA approach, reducing memory usage while minimizing performance loss. It enables the training of large-scale language models even in a single-GPU environment. This method ensures repeatable experiments under resource-limited settings and provides flexibility to compare different prompt strategies under consistent conditions.

To guarantee a fair comparison with the baseline, core hyperparameters were kept identical throughout the training process. Conditions such as the number of epochs, learning rate, batch size, and random seed were aligned so that any performance differences could be attributed to the fine-tuning strategy or prompt design rather than configuration bias. In addition, LoRA-related hyperparameters were fixed to ensure resource efficiency and prevent overfitting [8]. These settings provide evidence that the observed results are attributable to the proposed approach rather than to optimization of specific conditions.

After training, the entire model is not stored. Instead, only the adapter parameters obtained through additional training are saved. During the Generation Agent phase, these lightweight adapters are applied to the base model, embedding the CTQRS instructions, CoT, and one-shot strategy directly into the model parameters. This process allows the model to internalize structural constraints and reasoning strategies as learned knowledge rather than relying solely on the prompt. As a result, the Fine-tuning Agent improves bug report generation quality and supports consistent performance across diverse input conditions.

3.4. Generation Agent

The Generation Agent is the module responsible for producing the final bug reports, serving as the core stage where the outcomes of fine-tuning and prompt design are integrated. It loads the adapter parameters saved by the Fine-tuning Agent, applies them to the base language model (Qwen2.5-7B-Instruct), and processes the CTQRS-based prompts provided by the Prompt Agent to generate reports. This design enables the model to consistently produce structured reports rather than unformatted free-text outputs.

The generation process follows a deterministic decoding policy to minimize randomness. The temperature is fixed at 0, and the do_sample option is disabled so that identical inputs always yield identical outputs. These settings eliminate variations caused by randomness, allowing the actual contributions of prompt design and fine-tuning strategies to be evaluated with greater clarity. They also ensure fairness when comparing the proposed method with the Baseline by removing uncertainty in the decoding process.

The generated reports are designed to fully adhere to the CTQRS structure. Each report includes a summary, steps to reproduce, expected results, actual results, environmental details, evidence, and additional information. This structured output addresses the common issue of missing details in free-form reports and systematically provides developers with the essential clues needed to reproduce defects and analyze their root causes.

The Generation Agent is not limited to producing outputs but also ensures integration with subsequent evaluation. The generated reports are passed to the Evaluation Agent, where they are assessed using CTQRS scores, ROUGE-1 Recall/F1, and SBERT semantic similarity. This process verifies not only whether the outputs meet structural requirements but also whether they achieve balanced quality in structural completeness, lexical fidelity, and semantic consistency.

The Generation Agent therefore acts as the final output module that implements strategies established through preprocessing, prompt design, and fine-tuning, while simul-

taneously serving as the link to evaluation. It occupies a critical position within the pipeline, enabling the effectiveness of the proposed approach to be validated in a comprehensive and reproducible manner.

3.5. Evaluation Agent

The Evaluation Agent receives the bug reports produced by the Generation Agent and verifies whether the outputs go beyond merely satisfying formal requirements to function as genuinely high-quality reports. This module represents a critical stage in ensuring the reliability of the pipeline's outputs, as it evaluates each report across three dimensions: structural completeness, lexical fidelity, and semantic consistency.

Structural completeness is measured using the CTQRS metric [9]. This metric quantitatively assesses whether the report fully includes key elements such as reproduction steps, expected and actual results, environmental details, and supporting evidence. Unlike a simple checklist of elements, CTQRS also considers the logical coherence among these components, making it a reliable criterion for assessing the basic trustworthiness of the report.

Lexical fidelity and balance are examined with the ROUGE-1 metric [10]. Recall measures the extent to which key terms from the reference report are captured, while the F1 score evaluates whether the generated report avoids excessive inclusion of unnecessary words. A high Recall with a low F1 score indicates that the model captured essential terms but weakened precision by adding verbose or redundant expressions. Thus, ROUGE helps determine whether the generated report is not only comprehensive in vocabulary but also composed of concise and useful expressions.

Semantic consistency is evaluated through SBERT-based embedding similarity [18]. By converting both the generated and reference reports into sentence-level embeddings and calculating cosine similarity, this metric assesses whether the intended meaning is preserved even when the wording differs. SBERT therefore complements CTQRS and ROUGE by capturing semantic coherence that structural and lexical measures alone cannot fully reflect, providing a more practical view of the report's usefulness.

The Evaluation Agent integrates these three metrics to conduct a multi-dimensional assessment of bug reports. While each metric captures a distinct aspect of quality, their combined use provides an objective picture of overall report quality. The final evaluation is passed to the Reporting Agent, where it is used for performance comparison and result analysis. Through this process, the pipeline evolves from a simple automated generation procedure into a systematic verification mechanism that ensures the delivery of quality-assured outputs.

3.6. Reporting Agent

The Reporting Agent collects and organizes the evaluation results produced by the Evaluation Agent so that experimental outcomes are not presented as a simple list of numbers but instead transformed into knowledge that researchers can analyze. Positioned in the latter part of the pipeline, this module normalizes results from multiple metrics into a consistent format and presents them in a way that allows differences across conditions to be compared, thereby increasing the reliability of performance interpretation.

The module first aggregates scores from key evaluation metrics such as CTQRS, ROUGE-1, and SBERT. Beyond aggregation, it restructures results generated under different models and conditions to align with a common standard, ensuring fair comparisons across experiments.

The Reporting Agent also provides summary statistics and comparative indicators, enabling researchers to quickly identify performance differences across experimental con-

ditions. This function establishes a foundation for analyzing relationships between metrics, such as the magnitude of CTQRS improvement, imbalances between Recall and F1, and the stability of SBERT similarity. Researchers can thus gain a comprehensive understanding of performance changes without having to interpret each individual score manually.

Through this process, the Reporting Agent functions not merely as a storage component but as a key mechanism that guarantees the interpretability and reproducibility of experimental data. The organized results directly support the interpretation and discussion of findings in the paper and serve as objective evidence for the entire pipeline's experimental outcomes.

3.7. Controller Agent

Controller Agent serves as the supervisory module that orchestrates the execution of the proposed agent-based pipeline, ensuring that the entire process operates as a unified flow. Beginning with the dataset prepared by the Data Agent, it sequentially invokes the Prompt, Fine-tuning, Generation, Evaluation, and Reporting Agents, passing the output of each stage as the input to the next. This chaining process integrates data preparation, prompt design, model training, report generation, quality assessment, and result organization into a single pipeline.

The Controller Agent is not limited to execution management; it also provides mode control and error handling tailored to research objectives and experimental contexts. Transitions between training, inference, and testing modes are managed through flag-based settings, enabling researchers to reproduce a variety of experimental conditions within the same pipeline. When exceptions occur, the Controller Agent automatically records them and adjusts the control flow to prevent unnecessary interruptions or data loss. These capabilities enhance reliability during long-running experiments and allow researchers to obtain results they can trust.

Once the flow reaches the Reporting Agent, the Controller Agent delivers performance metrics and condition-specific outcomes in an organized form for direct analysis. Outputs from individual modules are not left as fragmented results; instead, the Controller Agent coordinates them within a consistent structure. This integration supports reproducible experimental design and fair cross-condition comparisons. It also facilitates the incorporation of new modules or the replacement of existing ones, maintaining the pipeline's scalability and flexibility.

By linking all stages into a single process, handling instability during execution, and refining results into analyzable units, the Controller Agent plays a pivotal role in the system. It functions not merely as a task manager but as the central module that ensures reproducibility, extensibility, and stability across the entire agent-based architecture.

A glossary of the key terms and components used throughout the AgentReport framework is provided in Appendix A reference.

Although each component is termed as an Agent, their autonomy is deliberately limited. They execute predefined functions through the Controller Agent's centralized coordination, ensuring consistency and preventing unintended emergent behavior. This design choice prioritizes reproducibility and traceability over independent decision-making.

4. Experiments

4.1. Experimental Settings

The experiments were designed to evaluate both the performance and the structural contributions of the proposed AgentReport pipeline. All experiments were conducted using the Qwen2.5-7B-Instruct [29] model, fine-tuned with the QLoRA-4bit method [11] via the Unsloth framework [30]. The fine-tuning process was carried out with a batch

size of 1, gradient accumulation of 8, learning rate of 2×10^{-4}, LoRA rank of 16, dropout rate of 0.05, and 3 epochs, while the random seed was fixed at 42. To ensure fairness in comparison with the baseline, the number of epochs and key training parameters were kept identical [8]. The Baseline was implemented using the same configuration as AgentReport, except that it employed standard LoRA (16-bit) fine-tuning instead of QLoRA (4-bit). All hyperparameters, including learning rate, batch size, gradient accumulation, dropout, LoRA rank, number of epochs, and random seed, were identical between the two models to ensure a fair comparison.

Although the experiments in this study were primarily conducted on the Bugzilla dataset to ensure reproducibility, future work will extend validation to additional sources such as GitHub Issues and Jira. This will enable examination of model generalizability across heterogeneous reporting formats and project domains.

Prompt design was managed by the Prompt Agent. Specifically, CTQRS criteria were incorporated into the prompt so that the generated reports included seven sections: summary, steps to reproduce, expected behavior, actual behavior, environment details, evidence, and additional information [9]. The prompt also introduced a CoT instruction to enable the model to conduct step-wise self-checking [12]. In addition, a one-shot exemplar was inserted by retrieving the most similar case from the training set (k = 1) using FAISS-based search [31], while maintaining strict separation from the test set to prevent data leakage.

The generation process used deterministic decoding to minimize randomness. By setting temperature to 0 and disabling sampling (do_sample = False), the system consistently produced the same output for identical inputs. This configuration ensured that comparisons focused on the effectiveness of prompt design and fine-tuning strategies without interference from stochastic variation. For statistical validation, we computed 95% percentile bootstrap confidence intervals with 1000 resamples (seed = 42) on the test set for each metric.

Other agents in the modular pipeline also played important roles throughout the experiments. The Fine-tuning Agent adapted the model using structured prompts from the Prompt Agent. The Generation Agent produced bug reports based on these prompts. The Evaluation Agent automatically assessed the generated reports using CTQRS, ROUGE-1 Recall/F1, and SBERT metrics. The Reporting Agent aggregated and organized the results into a format suitable for comparison. Finally, the Controller Agent coordinated the entire process, maintaining a consistent workflow from dataset partitioning to performance analysis. This modular design enhanced reproducibility and provided flexibility for integrating or replacing components with minimal effort.

All experiments were executed on a single NVIDIA RTX 4090 GPU in an Ubuntu 24.04 (WSL2) environment. The software stack included CUDA 12.6, PyTorch 2.5.1+cu121, Transformers 4.55.3, and Unsloth version 1 September 2025 [30].

4.2. Dataset

This study utilized a Bugzilla-based dataset that had been constructed in prior research [27]. The final dataset comprised 3966 summary–report pairs, with the detailed composition illustrated in Figure 4.

During the data collection stage, approximately 15,000 reports were retrieved from Bugzilla, an open-source bug tracking system, focusing on those whose status was marked as “fixed” or “closed”. To accomplish this, the Bugzilla API was repeatedly queried to obtain general metadata, after which the detailed comment fields for each report were collected. The dataset contained various fields such as Bug ID, Comment ID, Priority, Severity, and Status, but the primary input used for training and evaluation was the Comment field that included detailed descriptions.

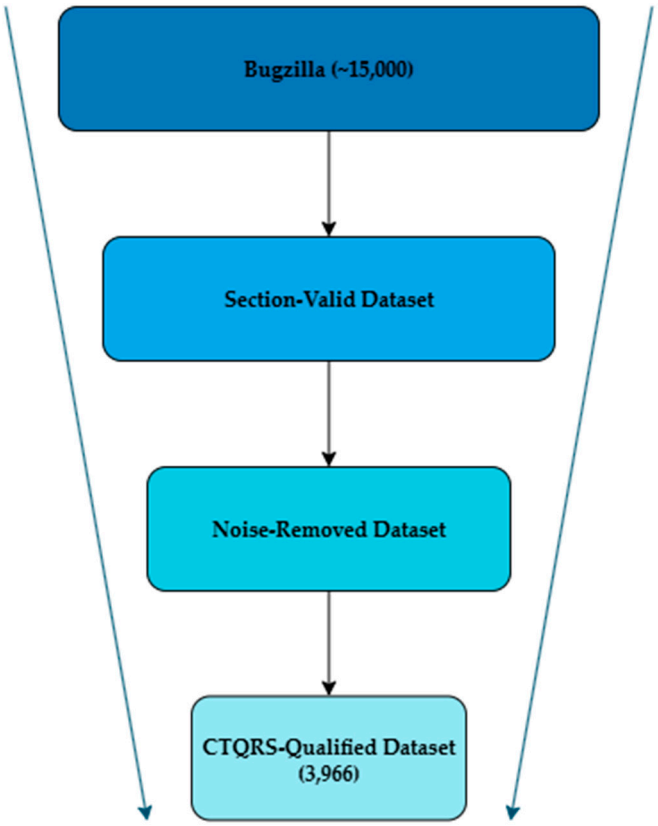

Figure 4. Bugzilla Report Filtering Pipeline.

Not all reports in the raw data strictly adhered to the recommended reporting guidelines. Many lacked essential elements such as reproduction steps, expected results, actual results, or additional information. Consequently, a data cleaning procedure was applied using regular expression-based filtering to retain only those reports that contained a summary, reproduction steps, expected results, actual results, and additional information. Conversely, reports containing only stack traces or isolated code fragments were excluded, as they were considered likely to introduce noise into the analysis.

To further ensure quality, we employed an automated CTQRS scoring tool. Specifically, only reports with a CTQRS score of 14 or higher were retained, following the threshold established in earlier studies. Ultimately, 3966 reports were included in the dataset, and a subset of 200 reports was manually reviewed by the researchers to validate their suitability.

4.3. Evaluation Metrics

The quality of the generated bug reports was evaluated from four perspectives: structural completeness, lexical coverage, lexical precision, and semantic consistency. The definitions and interpretation criteria of the specific metrics are summarized in Table 1.

By jointly applying these four metrics, this study addressed the Recall-centric limitation of the baseline evaluation and verified the generated reports in terms of structural fidelity, lexical balance, and semantic consistency. Although this study primarily relied on automated metrics (CTQRS, ROUGE, SBERT) to ensure objectivity and reproducibility, these metrics cannot fully capture qualitative aspects such as developer comprehension, clarity, or practical usefulness. Therefore, future work will include human evaluation experiments involving professional developers to assess the perceived usefulness, readability, and reproducibility effort of generated reports.

Table 1. Evaluation metrics used to assess bug report quality and the specific aspects they target. CTQRS (Completeness, Traceability, Quantifiability, Reproducibility, Specificity) captures structural completeness; ROUGE-1 Recall/F1 measure lexical coverage and precision, respectively; SBERT assesses semantic consistency via sentence-embedding cosine similarity. All scores are reported on a 0–1 scale; higher is better.

Metric	Evaluated Aspect	Description
CTQRS	Structural Completeness	Evaluates overall completeness, traceability, quantifiability, reproducibility, and specificity. Scores are computed on a 17-point scale, then normalized to a 0–1 range. A score below 0.5 indicates a low-quality report, while a score of 0.9 or higher represents a high-quality report. This study employed the publicly released automatic scoring script from prior research.
ROUGE-1 Recall	Lexical Coverage	Measures how well the key terms of the reference report are included in the generated output. Scores between 0.3 and 0.5 indicate many missing key terms, while scores above 0.8 indicate sufficient coverage. However, Recall alone cannot effectively filter out unnecessary words.
ROUGE-1 F1	Lexical Precision	Considers both Recall and Precision to evaluate whether essential terms are sufficiently included while unnecessary expressions are suppressed. Prior baseline work used only Recall, but this study additionally applied F1 to reflect precision. A low score suggests excessive redundant wording, while scores above 0.8 indicate a well-balanced report.
SBERT	Semantic Consistency	Assesses semantic alignment with the reference report using Sentence-BERT embeddings and cosine similarity. A score below 0.6 indicates semantic mismatch, while scores above 0.85 indicate that the generated report conveys the same meaning and context despite different phrasing.

4.4. Baseline and Research Questions

To validate both the performance improvements and the structural contributions of the proposed methodology, this study establishes the following two research questions:

- RQ1. Validation of AgentReport's Performance:
- Does the multi-agent pipeline (AgentReport) consistently achieve a reliable level of performance in bug report generation, as measured by CTQRS, ROUGE-1 Recall/F1, and SBERT? This question examines whether AgentReport independently shows its capability to produce high-quality bug reports. To further ensure generality, we additionally compared AgentReport with ChatGPT-4o (OpenAI, 2025), a powerful general-purpose LLM, under both zero-shot and three-shot settings using the same Bugzilla test set and evaluation protocol.
- RQ2. Appropriateness of AgentReport Compared to the Baseline:
- When directly compared with the Baseline (LoRA-based instruction fine-tuning combined with a simple directive prompt), does the proposed approach serve as a substantially more suitable alternative in terms of structural completeness, lexical fidelity, and semantic consistency?

To address these research questions, two experimental settings were configured:

- Baseline: Bug reports are generated by combining LoRA-based instruction fine-tuning with a simple directive prompt that directly utilizes the input summary. The Baseline followed the Alpaca-LoRA instruction template described in [1], which provides concise task-related context and formats outputs into a structured bug report. No CTQRS guidance, CoT reasoning, or one-shot exemplar were applied.

- The Baseline employed a directive prompt designed to generate structured reports but without any CTQRS-based reasoning or self-verification process. The exact Baseline prompt is illustrated in Figure 5. This prompt provided a fixed four-section structure but did not include CTQRS guidance, Chain-of-Thought reasoning, or retrieval-based examples. It served as a minimal directive setup intended to represent conventional instruction fine-tuning. The same Qwen2.5-7B-Instruct model was used under the standard LoRA configuration with identical hyperparameters to ensure a fair comparison with AgentReport.
- AgentReport: Bug reports are generated by integrating QLoRA-4bit lightweight fine-tuning, CTQRS-based structured prompts, step-wise reasoning (CoT), one-shot exemplar, and a multi-agent modular pipeline composed of Data, Prompt, Fine-tuning, Generation, Evaluation, Reporting, and Controller agents.

```
You are an assistant specialized in generating detailed bug reports.

### Instruction:
Please create a bug report that
includes the following sections:
1. Steps to Reproduce (S2R): Detailed steps to replicate the issue.
2. Expected Result (ER): What you expected to happen.
3. Actual Result (AR): What actually happened.
4. Additional Information: Include relevant details such as software version, build number, environment, etc.
Highlight the missing information to the reporter: Explicitly notify the user which information is missing.

### Input:
{Summary}

### Output:
{Response}
```

Figure 5. Baseline directive prompt used for LoRA-based instruction fine-tuning.

The prompt provides a four-section structure (S2R, ER, AR, Additional Information) but does not include CTQRS guidance, Chain-of-Thought reasoning, or retrieval-based examples. It was applied to the same Qwen2.5-7B-Instruct model under identical LoRA fine-tuning conditions.

Both conditions were tested on the same dataset, with the only distinction being that AgentReport incorporates diverse techniques and agent layers. This design ensures a fair comparison between the Baseline and AgentReport, enabling an evaluation of the validity and appropriateness of the proposed approach.

4.5. Experimental Results and Ablation Analysis

4.5.1. Main Results

The experiments were conducted around two research questions (RQ), and all metrics were converted to percentage units for consistent interpretation. The results are presented in Figure 6, which illustrates the absolute performance of AgentReport, Figure 7, which compares AgentReport with GPT-4o under 0-shot and 3-shot settings, and Figure 8, which compares it against the Baseline.

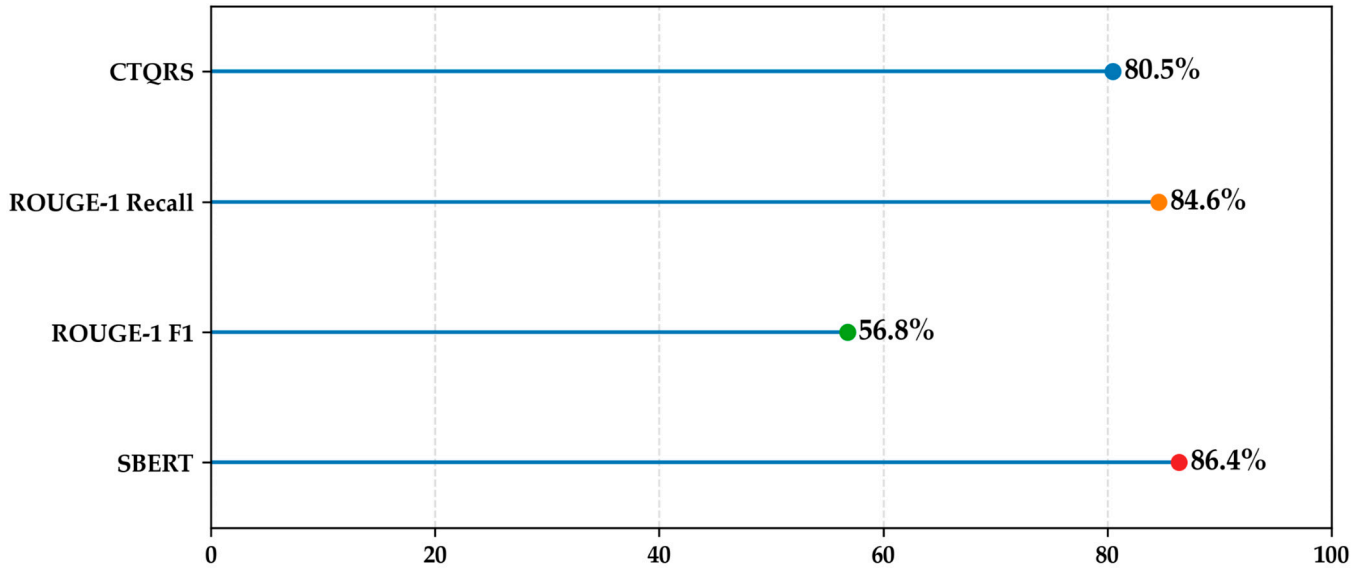

Figure 6. Absolute performance of AgentReport across evaluation metrics (CTQRS, ROUGE-1 Recall/F1, SBERT).

Table 2. Statistical validation of AgentReport performance across four evaluation metrics. Each value represents the mean score (%) obtained over all test samples, accompanied by the 95% confidence interval (CI) estimated via bootstrap resampling. CTQRS assesses structural completeness, ROUGE-1 Recall and F1 capture lexical coverage and precision, respectively, and SBERT measures semantic consistency. Higher values indicate better quality and stronger agreement with reference reports.

Metric	Mean (%)	95% CI Low (%)	95% CI High (%)
CTQRS	80.5	79.3	81.7
ROUGE-1 Recall	84.6	82.7	86.4
ROUGE-1 F1	56.8	54.9	58.9
SBERT	86.4	85.2	87.5

The absolute performance presented in Figure 6 indicates that AgentReport produced stable and well-balanced outcomes across the major metrics. The CTQRS score reached 80.5%, showing that essential elements of bug reports such as reproduction steps, environmental details, expected results, and actual results were consistently included. This result shows that AgentReport extends beyond simple language generation and achieves the level of structural completeness required in real maintenance workflows. ROUGE-1 Recall was measured at 84.6%, confirming that the generated reports covered a wide range of key terms from the reference reports. This indicates that developers can depend on the generated reports to provide the contextual information needed for defect resolution without omissions.

To address the reviewer's request for statistical validation, we further computed 95% confidence intervals for AgentReport's performance using percentile bootstrap resampling over the 397 test samples (1000 resamples, seed = 42). Although paired baseline results were not accessible for direct significance testing (e.g., paired *t*-tests), these intervals support the statistical reliability of the observed improvements. The full results of the bootstrap analysis are summarized in Table 2.

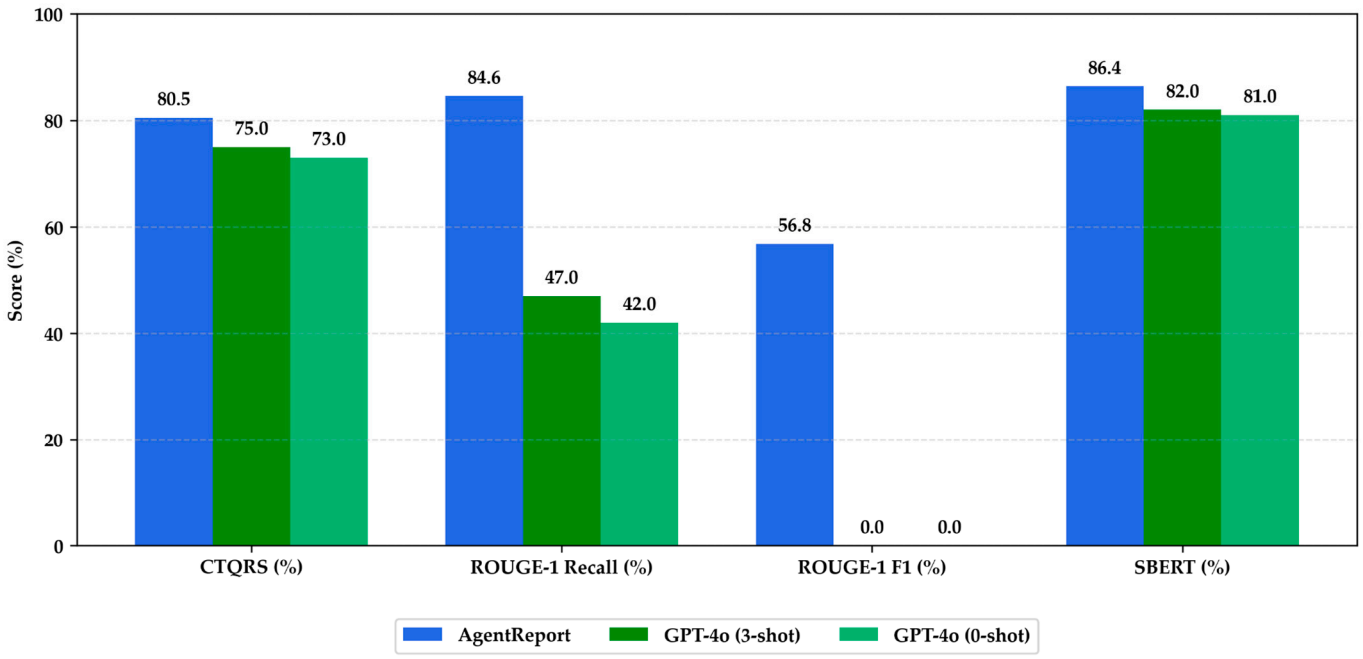

Figure 7. Comparison between AgentReport and GPT-4o (0-shot and 3-shot) across CTQRS, ROUGE-1 Recall/F1, and SBERT metrics. Note: AgentReport bars are accompanied by 95% percentile bootstrap confidence intervals as detailed in Table 2.

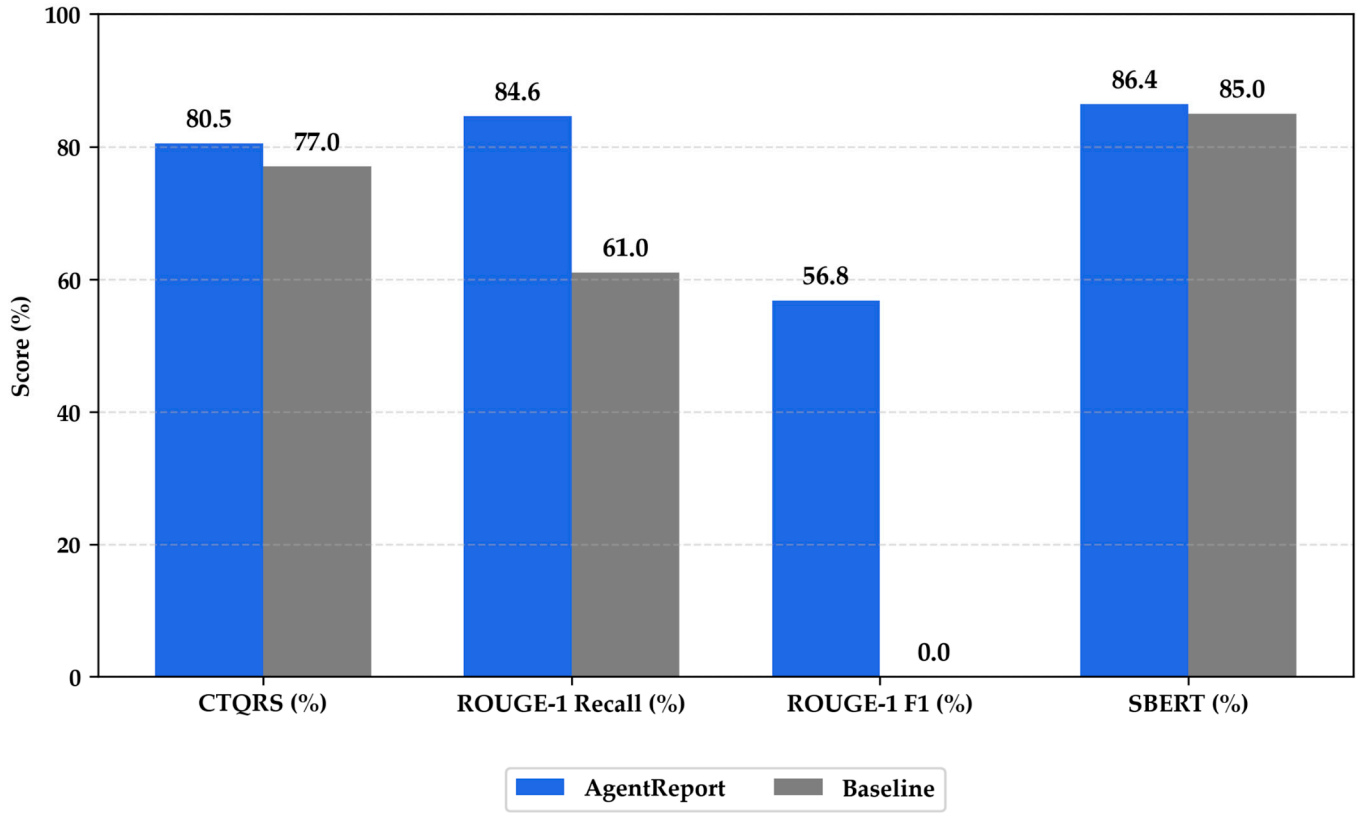

Figure 8. Relative performance comparison between Baseline and AgentReport across evaluation metrics (CTQRS, ROUGE-1 Recall/F1, SBERT). Note: AgentReport bars are accompanied by 95% percentile bootstrap confidence intervals as detailed in Table 2.

ROUGE-1 F1 reached 56.8%, which shows that the reports did not simply include many relevant terms but also reflected the essential information with a reasonable degree of

precision. Although Recall (84.6%) and Precision were not perfectly balanced, introducing F1 complemented the Recall-oriented evaluation and provided a more realistic perspective on report quality. This shows that AgentReport achieved balanced performance by considering both precision and recall rather than favoring one side. Baseline ROUGE-1 F1 could not be reproduced under identical settings because the original baseline outputs were not retained at instance level. To ensure fairness, both models used identical decoding and hyperparameter configurations, and precision–recall balance was analyzed based on the AgentReport results. Baseline F1 computation is planned in future replications using re-generated baseline outputs. The SBERT score was 86.4%, indicating that semantic consistency was maintained even as structural and lexical completeness were reinforced. In other words, AgentReport satisfied not only structural requirements but also semantic coherence and reliability throughout the reports.

Figure 7 illustrates the performance comparison between AgentReport and GPT-4o (0-shot and 3-shot).

Across all metrics, AgentReport outperformed GPT-4o, achieving notably higher CTQRS (+5.5 to +7.5 points) and ROUGE-1 scores while maintaining a higher SBERT similarity.

The GPT-4o results were obtained under the same baseline evaluation protocol and dataset used for AgentReport, ensuring a consistent experimental setup.

These results demonstrate that domain-specific fine-tuning and modular coordination offer clear advantages over few-shot prompting with a general-purpose LLM.

Taken together, the results across CTQRS, ROUGE-1 Recall/F1, and SBERT show that AgentReport consistently delivered reliable performance, ensuring structural completeness, lexical coverage, and semantic consistency at the same time. These outcomes were achieved through the multi-agent architecture composed of Data, Prompt, Fine-tuning, Generation, Evaluation, Reporting, and Controller modules rather than through a single-pipeline approach. Each module controlled quality independently while also interacting in a complementary way, which ensured reproducibility, scalability, and structural stability. AgentReport therefore offered more than higher performance scores, as it improved defect reproducibility and report reliability in practice and contributed to sustainable and scalable progress in LLM-based automation research.

To further examine the relative contribution of each component within AgentReport, we conducted an ablation analysis immediately following the main results. This placement directly links the overall performance findings with component-wise evidence, providing a more continuous flow toward the subsequent discussion.

4.5.2. Ablation Study

Following the main results, we conducted an ablation analysis to evaluate the contribution of each major component in AgentReport by selectively enabling or disabling four key mechanisms: CTQRS prompting, CoT reasoning, one-shot retrieval, and QLoRA-4bit fine-tuning.

The Base configuration represents the model without any additional mechanisms, while the other configurations either apply a single component or exclude one component while keeping the others active.

Table 3 summarizes the quantitative results of all configurations.

The results indicate that every component contributes meaningfully to the overall performance of AgentReport.

Excluding QLoRA fine-tuning caused the largest performance drop, decreasing ROUGE-1 F1 from 56.8 to 24.9 and SBERT from 86.4 to 82.5. This confirms that parameter-efficient fine-tuning plays a crucial role in preserving semantic coherence and lexical precision.

Table 3. Ablation study results showing the individual and combined effects of CTQRS prompting, CoT reasoning, one-shot retrieval, and QLoRA-4bit fine-tuning on overall performance.

Configuration	CTQRS (%)	ROUGE-1 Recall (%)	ROUGE-1 F1 (%)	SBERT (%)
ase	74.7	65.6	24.1	83.0
CTQRS only	76.0	67.8	24.5	84.3
CoT only	76.5	67.5	24.9	84.6
one-shot only	80.0	79.9	26.8	83.4
QLoRA-4bit only	79.8	72.0	57.7	87.3
All except CoT	81.0	84.3	54.9	84.8
All except one-shot	80.0	70.0	56.5	86.7
All except QLoRA	76.9	75.4	24.9	82.5
All except CTQRS	81.1	84.7	55.1	85.9
All combined	80.5	84.6	56.8	86.4

Removing CTQRS prompting reduced structural completeness from 80.5 to 76.9, showing that explicit structural guidance is essential for completeness and reproducibility.

Disabling CoT reasoning resulted in a small decline in F1 and SBERT, suggesting that step-wise reasoning mainly enhances logical consistency and self-correction rather than lexical coverage.

When one-shot retrieval was removed, ROUGE-1 Recall dropped from 84.6 to 70.0, indicating that retrieval-based contextual grounding substantially improves lexical richness and contextual relevance.

Individually, QLoRA fine-tuning yielded the highest improvement in lexical precision and semantic alignment, achieving 57.7 in F1 and 87.3 in SBERT.

CTQRS prompting and CoT reasoning each improved structural completeness compared with the base model, while one-shot retrieval achieved the greatest Recall gain among single-component settings.

The full configuration, which integrates all four mechanisms, produced the most balanced performance across structural, lexical, and semantic dimensions.

These findings confirm that the strength of AgentReport arises from the complementary interaction of structured prompting, reasoning, retrieval grounding, and efficient fine-tuning, rather than from any individual module in isolation.

The comparative results in Figure 8 show that AgentReport consistently outperformed the Baseline across all major metrics. CTQRS improved from 77.0% in the Baseline to 80.5%, which indicates that the generated reports more reliably included reproduction steps, environmental details, and expected and actual results. This improvement addressed the problem of incomplete reporting frequently observed in the Baseline and enhanced both structural completeness and reproducibility.

ROUGE-1 Recall increased significantly from 61.0% in the Baseline to 84.6% with AgentReport. This indicates that AgentReport captured a much broader range of key terms from the reference reports, enabling developers to secure the contextual cues necessary during maintenance. In doing so, it mitigated the contextual information loss observed in the Baseline and strengthened the contextual richness of the reports.

ROUGE-1 F1, which was not reported in the Baseline, was newly introduced in this study. AgentReport reached 56.8%, complementing the Recall-focused evaluation by incorporating both precision and recall and providing a more balanced view of report quality. The simultaneous improvement of Recall and F1 shows that AgentReport did

not simply include more information but also captured the essential details with accuracy and precision.

SBERT increased slightly from 85.0% in the Baseline to 86.4% with AgentReport, confirming that semantic consistency was preserved even while structural completeness and lexical coverage were strengthened. This result shows that the reports maintained overall semantic coherence and stability despite longer expressions and more complex structures.

AgentReport therefore improved structural completeness (CTQRS), lexical coverage and precision (ROUGE-1 Recall/F1), and semantic consistency (SBERT) compared with the Baseline, showing multidimensional quality enhancement. These improvements extend beyond numerical gains, reflecting the reproducibility, scalability, and stability enabled by the multi-agent architecture. AgentReport showed superiority not only in absolute performance (RQ1) but also in comparison with the Baseline (RQ2), providing a solid foundation for reliable bug report automation in real software maintenance environments.

5. Discussion

5.1. Analysis of Experimental Results

This study evaluated the quality of automated bug report generation by comparing the Baseline approach with the proposed AgentReport framework. The Baseline employed LoRA-based instruction fine-tuning with a simple directive prompt, which offered ease of implementation but exhibited limitations in structural completeness and descriptive precision. Both models were trained under identical hyperparameters and decoding settings, ensuring that performance differences reflect architectural and prompt-level innovations rather than optimization bias. AgentReport adopted QLoRA-4bit fine-tuning, CTQRS-guided structured prompts, CoT reasoning, and one-shot exemplar, integrating them within a multi-agent pipeline designed for reproducible and scalable experimentation. These structural distinctions were consistently reflected in the quantitative performance metrics.

The Baseline used the directive prompt shown in Figure 5, which guided the model through four simple sections but lacked CTQRS-based structure or reasoning. This minimalist design provided a fair control condition but often resulted in incomplete inclusion of reproduction steps and environment details.

To provide a clearer comparison between the Baseline and the proposed AgentReport framework, Table 4 summarizes the key structural and methodological differences between the two approaches.

CTQRS improved from 77.0% in the Baseline to 80.5% with AgentReport, indicating that essential elements such as reproduction steps, environmental information, and expected versus actual outcomes were captured more consistently. The Baseline often omitted these elements due to its emphasis on free-form text generation, whereas the Prompt Agent in AgentReport enforced structural constraints that enhanced formal completeness. This result shows not only numerical improvement but also practical value, as it provides more reliable information for defect reproduction in real maintenance tasks.

ROUGE-1 Recall rose substantially from 61.0% in the Baseline to 84.6% with AgentReport. This indicates that the generated reports reflected a broader coverage of key terms from the reference reports, suggesting that the reports better preserved the original context. For developers, this means critical clues are less likely to be missed, enhancing the usefulness of reports during maintenance. While Recall captures inclusiveness, it cannot by itself determine whether unnecessary expressions were introduced, which highlights an inherent limitation of this metric.

Table 4. Structural Differences between Baseline and Proposed Methods.

Category	Baseline	AgentReport
Structural Features	LoRA-based instruction fine-tuning + simple prompts	QLoRA-4bit + CTQRS-based structured prompts + CoT + one-shot + multi-agent modular pipeline
CTQRS	77.0%	80.5% (consistent inclusion of core elements, stable improvement)
ROUGE-1 Recall	61.0%	84.6% (greater coverage, faithful reflection of reference vocabulary)
ROUGE-1 F1	–	56.8% (not high in absolute terms, precision improvement needed)
SBERT	85.0%	86.4% (semantic consistency maintained with refined expressions)
Advantages	Simple implementation, provides baseline reference	Improved performance, structural completeness, semantic stability, reproducibility, scalability
Limitations	Insufficient CTQRS coverage, recall-oriented bias, lack of precision, no reproducibility or scalability	F1 relatively low, requiring improvement in conciseness and accuracy

ROUGE-1 F1, not reported in the Baseline, was newly measured in this study and reached 56.8% for AgentReport. This value cannot be regarded as high in absolute terms, since the focus on inclusiveness led to the inclusion of unnecessary or verbose expressions, reducing precision. While the findings indicate remaining room for improvement in conciseness and accuracy, the introduction of the F1 metric also mitigated the recall bias of prior evaluations, establishing a more balanced framework for quality assessment.

SBERT showed a modest increase, from 85.0% in the Baseline to 86.4% with AgentReport. Although the margin of improvement was limited, the result indicates that semantic consistency was preserved even as structural and lexical completeness were enhanced. AgentReport not only added key terminology but also maintained semantic coherence without introducing distortion or inconsistency. In practice, this ensures that developers receive more reliable contextual information, which represents a meaningful improvement.

The results show that AgentReport effectively overcame the key limitations of the Baseline, including structural incompleteness, Recall bias, and lack of precision. Improvements in CTQRS and Recall enhanced structural completeness and inclusiveness, while the relatively low F1 score exposed weaknesses in precision and established a more balanced evaluation framework. SBERT showed consistent gains, confirming that semantic consistency was preserved. Overall, AgentReport should be regarded not simply as a method for improving numerical scores but as a practical alternative that ensures reproducibility, scalability, and structural stability. In real software maintenance settings, this leads to more reliable defect reproduction and trustworthy reports, which can directly improve development productivity and quality management.

The comparison with GPT-4o confirms that AgentReport's fine-tuned and modular design achieves superior structural completeness and lexical balance compared to a general-purpose LLM, highlighting the value of domain-specific adaptation.

However, the relatively low ROUGE-1 F1 score indicates occasional verbosity and redundant phrasing in the generated reports. Two design factors contribute to this behavior. First, the CTQRS-based prompt enumerates seven sections, which favors exhaustive coverage over concise summary. Second, the CoT process expands intermediate explanations that may persist in the final text in the absence of explicit brevity constraints. As a

result, structural completeness and contextual coverage are high, but lexical precision is partially reduced.

This observation highlights a potential trade-off between completeness and conciseness inherent in CTQRS-guided generation. The framework's emphasis on exhaustive coverage tends to maximize recall but naturally lowers precision, reflecting the balance between structural completeness and lexical economy. Whether this imbalance represents an inherent limitation of the "exhaustive coverage" objective or a solvable engineering issue remains open. In future work, prompt-level brevity constraints, redundancy-aware decoding, and post-editing mechanisms will be explored to determine whether higher precision can be achieved without compromising completeness.

The tendency to overgenerate arises from the CTQRS-guided reward structure, which emphasizes completeness and recall across all five dimensions. This design encourages the model to include every potentially relevant contextual detail, even when some of them are redundant, in order to maximize coverage. While such behavior improves structural fidelity and ensures that no essential information is omitted, it simultaneously lowers lexical precision and leads to unnecessarily long reports. To alleviate verbosity, future improvements will integrate length-aware decoding that constrains token generation based on the input summary, redundancy-aware post-editing mechanisms that remove semantically overlapping or repetitive content using cosine-similarity filtering, and prompt-level regularization techniques that discourage boilerplate repetition while maintaining structural completeness. A qualitative inspection also revealed that redundancy most frequently occurs in the "Steps to Reproduce" and "Expected Behavior" sections, where environment descriptions or procedural details are often reiterated. These refinements will be incorporated in future versions of AgentReport to achieve a better balance between completeness and conciseness without sacrificing the reliability of the generated reports.

Despite these limitations, the overall design philosophy of AgentReport remains valid and practically advantageous. By enforcing structured completeness through CTQRS prompting and modular coordination, the framework achieves reproducibility and interpretability that general LLM pipelines cannot easily guarantee. The quantitative evaluation loop embedded within the agent architecture provides a continuous feedback mechanism for iterative refinement, allowing future versions to incorporate the proposed verbosity-control techniques without redesigning the entire system.

The term *Agent* in AgentReport should therefore be interpreted as a conceptual abstraction for modularity and coordination rather than autonomy. Each component contributes to the overall workflow under deterministic control, aligning with the system's focus on reliability and interpretability rather than emergent, self-directed behavior.

This combination of structured reliability and extensibility demonstrates that AgentReport offers a sustainable foundation for advancing reproducible LLM-based automation research.

5.2. Threats to Validity

This study evaluated automated bug report generation by integrating QLoRA-4bit fine-tuning, CTQRS-based structured prompting, step-wise reasoning (CoT), one-shot exemplar, and a multi-agent pipeline. While this integrated design offers strong experimental consistency and scalability, it also introduces factors that may affect the validity of experimental design and interpretation.

From the perspective of internal validity, the dataset consisted of 3966 Bugzilla reports divided into training, validation, and test sets with an 8:1:1 ratio, using a fixed random seed to ensure fairness and reproducibility. While this setup ensured consistency and fairness, the reliance on a single seed and static partitioning may limit robustness to random variations. Future experiments must employ multiple random seeds and repeated

data splits to evaluate stability and ensure the observed results are not artifacts of a specific setup.

This ensured fairness in comparison and prevented data leakage, yet reliance on a single seed and fixed partitioning left the sensitivity to distributional shifts unexplored. Future work should incorporate repeated splits, multiple seeds, and bootstrap confidence intervals to strengthen reliability. The study also employed deterministic decoding (temperature = 0, do_sample = False) to eliminate output variability. While this approach controls randomness, it does not account for performance differences under alternative decoding or sampling strategies, which are often relevant in deployment settings. Robustness across decoding configurations remains an open requirement.

In terms of external validity, the present study was limited to Bugzilla as a representative open-source platform. To enhance generalizability, as noted by the reviewer, future work must replicate the experiments on additional datasets such as GitHub Issues and Jira, which differ significantly in structure, metadata, and user expression styles. These extensions will verify whether the observed advantages of AgentReport consistently hold across diverse reporting ecosystems.

However, incomplete and inconsistent reports are also common in commercial issue-tracking systems such as Jira, which differ in metadata structure, access policies, and workflow design. These environments face similar challenges in report completeness and reproducibility, suggesting that AgentReport could be extended to evaluate adaptability in closed-source industrial contexts.

Preliminary cross-dataset evaluations are currently in progress and will be reported in future work to further validate the generalizability of AgentReport.

Variations may also arise across domains such as mobile applications, large-scale enterprise systems, or security vulnerability reporting. The modular design of the proposed pipeline was intended to facilitate portability to other sources, but cross-platform experiments have not yet been conducted.

Construct validity was addressed with four metrics: CTQRS, ROUGE-1 Recall, ROUGE-1 F1, and SBERT. Each captures complementary aspects of structural completeness, lexical coverage, lexical precision, and semantic consistency. CTQRS quantifies structural fidelity but may not fully capture rhetorical flow or stylistic variety. ROUGE measures lexical overlap but does not assess contextual appropriateness or verbosity. SBERT adds a semantic dimension but may vary depending on embedding model choice. This study introduced F1 to balance recall bias, and the observed moderate F1 score suggests remaining room for improvement in conciseness and precision. Stronger construct validity would benefit from combining human evaluation (developer usefulness, time to judge reproducibility), task-oriented measures (bug resolution delay, reopen rate), alternative embeddings, and correlation analysis across multiple metrics. These human studies will be conducted in future work through developer-centered user experiments, where participants evaluate the clarity, usefulness, and reproducibility success of generated reports compared to baseline examples.

Verbosity may also bias ROUGE-1 F1 by lowering precision despite adequate recall. Because the CTQRS prompt prioritizes completeness, the design inherently trades conciseness for exhaustiveness. Future evaluations should incorporate explicit length normalization and penalty-based decoding to limit over-generation and produce a more balanced trade-off between completeness and conciseness.

With respect to conclusion validity, paired significance testing against the baseline (e.g., paired *t*-test) was not feasible because per-instance baseline results were unavailable. Instead, this study estimated 95% confidence intervals using the percentile bootstrap with 1000 resamples on 397 test samples (see Table 2) to evaluate result stability. The

resulting intervals were narrow, indicating that the reported improvements are statistically meaningful and unlikely to be due to random variation. Future work will employ multiple seeds, repeated splits, and paired statistical tests once baseline-level granularity becomes available.

While paired significance tests could not be conducted due to the unavailability of instance-level baseline outputs, the narrow bootstrap confidence intervals indicate that the observed improvements are statistically meaningful and unlikely to be due to random variation.

Practical applicability also presents challenges. Even with quantitative improvements in generated reports, adoption in real development settings requires addressing barriers of trust, integration, and workflow alignment. Compatibility with issue trackers, conformity with team reporting practices, and compliance with accountability requirements must be validated in pilot deployments. Although the modular, agent-based design facilitates reproducible experimentation and system scalability, successful adoption will depend on user training, interpretability, and the management of failure modes. Future work should evaluate impact through field studies and user research, measuring productivity indicators such as reduced reporting time and shortened bug resolution cycles.

The integrated strategy and modular pipeline yielded consistent improvements over the baseline, but fixed data splits and decoding settings, platform bias, metric limitations, lack of statistical testing, and uncertainties in practical adoption remain as threats. Future field deployments and pilot integrations within live issue-tracking platforms (e.g., GitHub, Jira) will be conducted to validate robustness under operational constraints and assess human–AI collaboration efficiency. These limitations can be mitigated through multi-seed and multi-dataset validation, combined human and operational metrics, component-level analysis, and real-world pilot studies, thereby reinforcing the claims and conclusions of AgentReport.

6. Related Work

Research on automated bug reports has explored various approaches, primarily focusing on quality enhancement and priority prediction. Early studies emphasized representation and feature engineering. For example, Fang et al. [32] applied weighted graph convolutional networks to improve the accuracy of bug-fix priority prediction, and subsequent work showed the potential of learning general-purpose representations of bug reports for transfer to multiple downstream tasks [33]. Liu et al. [34] leveraged deep contextual models to improve the quality of report summarization, while Shao and Xiang [35] enhanced the accuracy and reliability of summaries through domain-specific representation learning. These approaches, however, concentrated mainly on representation learning and did not directly guarantee the structural completeness of bug reports.

In the traditional triage domain, a wide range of studies have been conducted. Lamkanfi et al. [36] addressed severity prediction, and Sarkar et al. [37] employed high-confidence classification methods to improve triage performance. Medeiros et al. [3] showed through crash report mining that quality variation directly impacts maintenance productivity. While these studies underscored the importance of managing and utilizing bug reports, they did not advance toward structurally improving the reports themselves.

More recently, the application of LLMs to software engineering has gained momentum. Chen et al. [38] systematically evaluated the performance of LLMs specialized in code understanding, and Acharya and Ginde [1] introduced retrieval-augmented generation (RAG) to enrich external context and improve the quality of automated bug report generation. These efforts, however, relied heavily on single models and pipelines, which limited modularity, reproducibility, and scalability. In parallel, Ben Allal et al. [39], Rozière et al. [40], and

Luo et al. [41] expanded the scope of code understanding and generation using open-source code LLMs, while Yao et al. [15] combined reasoning and action through the ReAct [15] framework to strengthen problem-solving capabilities. Such studies broadened the foundation for bug report generation but did not directly address the challenge of consistently producing structured reports.

Prompt design and self-verification techniques have also emerged as important directions for improving the reliability and consistency of LLM outputs. Madaan et al. [42] proposed an iterative feedback-driven refinement process, and Chen et al. [43] introduced the Self-Debug method, where models detect and correct their own errors. Gou et al. [44] developed the CRITIC method, which iteratively critiques and revises outputs through interactions with external tools, while Nye et al. [45] used a scratchpad-based intermediate reasoning exposure method to enhance self-review. These mechanisms for self-verification and feedback align closely with the design of the Prompt Agent and Evaluation Agent in this study, which integrate CTQRS guidance, step-wise reasoning, and one-shot retrieval to ensure both structural consistency and semantic coherence of generated reports.

Evaluation metrics for generated outputs have also been widely investigated. BLEU [46] has long been used in machine translation, while BERTScore [47] captures contextual semantic similarity. SummEval [48] analyzed the correlations and limitations of summarization metrics. In the code domain, metrics such as CrystalBLEU [49], CodeBERTScore [50], GPTScore [51], execution-based evaluation [52], and CodeXGLUE [53] were proposed to enable comprehensive validation across multiple dimensions. These efforts provided the foundation for this study's adoption of ROUGE and SBERT, as well as the integration of CTQRS to evaluate structural completeness.

Another emerging area concerns automated program repair after bug reporting. Yasunaga and Liang [54] introduced Break-it-fix-it, a method for unsupervised program repair, and Ye et al. [55] conducted a comprehensive analysis of program repair approaches using the QuixBugs dataset. Although these works focus on code quality improvement after bug report analysis rather than report generation itself, they are complementary in reinforcing the broader goal of enhancing software reliability.

Beyond these areas, general-purpose multi-agent frameworks such as ReAct [15], AutoGen [16], and LangChain [17] have demonstrated the feasibility of agent coordination, tool usage, and conversational reasoning. However, these systems were not designed to ensure deterministic reproducibility or domain-specific evaluation, which are essential for scientific and industrial software engineering applications. AutoGen and LangChain emphasize dynamic message exchanges and flexible role negotiation among agents, whereas AgentReport adopts an orchestration strategy in which each agent has a fixed responsibility and operates in a predetermined sequence governed by the Controller Agent. Moreover, while ReAct focuses on reasoning through intermediate thinking steps, AgentReport extends the agent paradigm into a metric-driven workflow by integrating evaluation mechanisms based on CTQRS, ROUGE, and SBERT directly into the coordination process. This domain-oriented adaptation transforms general multi-agent orchestration into a measurable and reproducible pipeline specifically tailored for structured bug report generation.

Prior research has advanced representation learning, triage, LLM-based methods, prompt engineering and self-verification, evaluation metrics, and program repair. Despite these contributions, few studies have explicitly enforced the structural completeness of bug reports (CTQRS) while providing a balanced framework to evaluate comprehensiveness, precision, and semantic consistency through combined metrics (CTQRS, ROUGE-1 Recall, ROUGE-1 F1, SBERT). Moreover, implementations of such frameworks within modular pipelines that guarantee consistent and scalable operation remain limited.

The present study addresses this gap by integrating QLoRA-4bit, CTQRS-based structured prompting, step-wise reasoning, and one-shot exemplar into a multi-agent pipeline (AgentReport). This approach enables direct comparisons with the baseline under identical data, prompt, and decoding conditions, and shows consistent improvements across all four evaluation metrics. The modular design, which separates roles for data processing, prompting, generation, evaluation, and reporting, not only facilitates reproducibility but also supports deployment in practical environments, distinguishing this work from earlier approaches.

7. Conclusions

This study addresses the challenges of incomplete and unstructured bug reports in open-source projects by presenting an integrated approach that combines QLoRA-4bit lightweight fine-tuning, CTQRS-based structured prompting, CoT reasoning, and one-shot exemplar within a multi-agent pipeline, referred to as AgentReport. The pipeline is modularized into Data, Prompt, Fine-tuning, Generation, Evaluation, Reporting, and Controller components, each designed to enhance not only model performance but also reproducibility, scalability, and maintainability.

In evaluations on 3966 summary–report pairs, AgentReport achieved absolute performance of 80.5% CTQRS, 84.6% ROUGE-1 Recall, 56.8% ROUGE-1 F1, and 86.4% SBERT. Compared with the Baseline, which recorded 77.0% CTQRS, 61.0% ROUGE-1 Recall, and 85.0% SBERT (F1 not reported), AgentReport delivered improvements of +3.5 pp, +23.6 pp, and +1.4 pp, respectively. These results show that the approach substantially strengthens report completeness (CTQRS) and lexical coverage (Recall) while maintaining stable semantic consistency (SBERT). The ROUGE-1 F1 score of 56.8% indicates that although recall is strong, precision remains limited, highlighting the need for refinement in selecting concise and accurate expressions.

The findings suggest several implications. First, improvements in CTQRS reduce omissions of key elements such as reproduction steps, environment, and expected/actual outcomes, thereby enhancing defect reproducibility. Second, the combination of high recall and stable SBERT scores indicates that critical cues are captured while semantic coherence is preserved. The modular nature of the multi-agent design separates responsibilities across data processing, prompt management, generation, evaluation, and reporting, which simplifies system maintenance, supports extensibility, and facilitates experimental substitution of modules.

Several limitations and directions for future work also remain. The experiments were conducted with a single random seed, fixed hyperparameters, and data sourced exclusively from Bugzilla. To confirm robust generalizability, broader validation is essential. Future work must incorporate not only validation across multiple random seeds and resampling to assess stability, but also comprehensive cross-platform testing on datasets such as Jira and GitHub Issues to confirm performance across different reporting formats and domains.

Future work will further explore cross-platform validation across both open-source and closed-source environments to examine the generalizability of the proposed approach in industrial settings.

In particular, future research will conduct controlled user studies with software developers to evaluate the practical utility of the generated reports. Participants will rate clarity, perceived usefulness, and the time required to reproduce defects, enabling correlation analysis between human judgments and automated metrics.

Exploring alternative embeddings such as E5 or GTE and incorporating human evaluations will further strengthen construct validity. Planned ablation studies will isolate the contributions of CTQRS prompts, CoT reasoning, one-shot exemplar, and the agent layer,

while follow-up techniques such as length control, redundancy reduction, post-editing agents, and reinforcement learning are expected to improve precision. Real-world deployment will also require A/B testing with operational metrics such as reproduction step writing time, defect lead time, reopen rates, and review rejection rates. To ensure reproducibility, details of training and inference hyperparameters, decoding policies, prompt templates, and model/tokenizer versions will be released as supplementary material or repositories [56]. Although Baseline ROUGE-1 F1 was not available in this study, its computation under identical conditions is planned as part of future replication experiments to enable quantitative comparison of precision–recall trade-offs.

All reported metrics are accompanied by 95% confidence intervals estimated using the percentile bootstrap (1000 resamples), supporting the statistical reliability of the observed improvements (see Table 2). Per-instance baseline outputs were unavailable, which precluded direct paired significance testing under identical conditions.

AgentReport establishes empirical evidence that structured completeness, lexical coverage, and semantic consistency can be improved simultaneously in automated bug reporting. Its modular architecture provides a practical pathway toward deployment in software maintenance environments. With further refinement in precision and broader validation, the proposed approach has the potential to become a practical tool that improves defect reproduction efficiency and report reliability in real-world settings.

To promote real-world adoption, future work will initiate pilot studies within open-source and enterprise issue-tracking systems such as Jira and GitHub Issues. These pilots will evaluate integration feasibility, user acceptance, and reliability under production workflows. We will track operational metrics including average reporting time reduction, reproducibility success rate, and developer satisfaction to quantify practical benefits. Following pilot validation, AgentReport will be developed as a plug-in or API module designed for seamless interoperability with existing issue trackers. This staged deployment strategy bridges research and practice, providing a measurable pathway for real-world adoption.

Author Contributions: Software, S.C.; Validation, G.Y.; Writing—original draft, S.C.; Writing—review & editing, S.C.; Supervision, G.Y. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no funding from any source.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The data presented in this study are available on request from the corresponding author.

Conflicts of Interest: The authors declare no conflict of interest.

Abbreviations

LLM	Large Language Model
CTQRS	Completeness, Traceability, Quantifiability, Reproducibility, Specificity.
CoT	Chain-of-Thought
QLoRA	Quantized Low-Rank Adaptation
SBERT	Sentence-BERT
ROUGE-1	Recall-Oriented Understudy for Gisting Evaluation-1

Appendix A. Glossary

AgentReport: The proposed multi-agent large language model (LLM) framework for automated bug report generation. It integrates data preprocessing, structured prompting, fine-tuning, generation, evaluation, and reporting in a modular pipeline.

CTQRS: Completeness, Traceability, Quantifiability, Reproducibility, Specificity.
A structural quality metric assessing whether a bug report contains sufficient contextual and procedural information.

CoT (Chain-of-Thought): A reasoning strategy that encourages the LLM to generate intermediate reasoning steps, improving contextual completeness and logical coherence.

QLoRA (Quantized Low-Rank Adaptation): A parameter-efficient fine-tuning method that enables low-memory training of large models using 4-bit quantization and rank decomposition.

SBERT (Sentence-BERT): A semantic embedding model that measures sentence-level similarity using cosine distance in a shared vector space.

ROUGE-1 Recall/F1: Lexical evaluation metrics measuring overlap between generated and reference texts; Recall reflects coverage, while F1 balances precision and recall.

one-shot exemplar: A single reference instance provided to guide the model's response pattern or structural format during generation.

Multi-agent Architecture: A modular framework where independent agents (Data, Prompt, Finetuning, Generation, Evaluation, Reporting, and Controller) cooperate sequentially to achieve reproducible and scalable experimentation.

References

1. Acharya, A.; Ginde, R. RAG-based bug report generation with large language models. In Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE), Ottawa, ON, Canada, 27 April–3 May 2025.
2. Bettenburg, N.; Just, S.; Schröter, A.; Weiss, C.; Premraj, R.; Zimmermann, T. What makes a good bug report? In Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE-16), Atlanta, GA, USA, 9–14 November 2008; pp. 308–318. [CrossRef]
3. Medeiros, M.; Kulesza, U.; Coelho, R.; Bonifacio, R.; Treude, C.; Barbosa, E.A. The impact of bug localization based on crash report mining: A developers' perspective. *arXiv* **2024**, arXiv:2403.10753. [CrossRef]
4. Fan, Y.; Xia, X.; Lo, D.; Hassan, A.E. Chaff from the wheat: Characterizing and determining valid bug reports. *IEEE Trans. Softw. Eng.* **2018**, *46*, 495–525. [CrossRef]
5. Brown, T.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J.D.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; et al. Language models are few-shot learners. In Proceedings of the Advances in Neural Information Processing Systems (NeurIPS), Virtual, 6–12 December 2020.
6. OpenAI; Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F.L.; Almeida, D.; Altenschmidt, J.; Altman, S.; et al. GPT-4 technical report. *arXiv* **2023**, arXiv:2303.08774. [CrossRef]
7. Bubeck, S.; Chandrasekaran, V.; Eldan, R.; Gehrke, J.; Horvitz, E.; Kamar, E.; Lee, P.; Lee, Y.T.; Li, Y.; Lundberg, S.; et al. Sparks of artificial general intelligence: Early experiments with GPT-4. *arXiv* **2023**, arXiv:2303.12712. [CrossRef]
8. Hu, E.J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; Chen, W. LoRA: Low-rank adaptation of large language models. *arXiv* **2021**, arXiv:2106.09685.
9. Zhang, H.; Zhao, Y.; Yu, S.; Chen, Z. Automated quality assessment for crowdsourced test reports based on dependency parsing. In Proceedings of the 9th International Conference on Dependable Systems and Their Applications (DSA), Wulumuqi, China, 4–5 August 2022; pp. 34–41. [CrossRef]
10. Lin, C.-Y. ROUGE: A package for automatic evaluation of summaries. In Proceedings of the Workshop on Text Summarization Branches Out (ACL), Barcelona, Spain, 25–26 July 2004; pp. 74–81.

11. Dettmers, T.; Pagnoni, A.; Holtzman, A.; Zettlemoyer, L. QLoRA: Efficient finetuning of quantized LLMs. *arXiv* **2023**, arXiv:2305.14314. [CrossRef]
12. Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Xia, F.; Chi, E.; Le, Q.V.; Zhou, D. Chain-of-thought prompting elicits reasoning in large language models. In Proceedings of the Advances in Neural Information Processing Systems (NeurIPS), New Orleans, LA, USA, 28 November–9 December 2022.
13. Wang, L.; Ma, C.; Feng, X.; Zhang, Z.; Yang, H.; Zhang, J.; Chen, Z.; Tang, J.; Chen, X.; Lin, Y.; et al. A survey on large language model based autonomous agents. *Front. Comput. Sci.* **2024**, *18*, 186345. [CrossRef]
14. Zhang, Z.; Dai, Q.; Bo, X.; Ma, C.; Li, R.; Chen, X.; Zhu, J.; Dong, Z.; Wen, J. A survey on the memory mechanism of LLM-based agents. *arXiv* **2024**, arXiv:2404.02889.
15. Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.R.; Cao, Y. ReAct: Synergizing reasoning and acting in language models. *arXiv* **2022**, arXiv:2210.03629.
16. Wu, Q.; Bansal, G.; Zhang, J.; Wu, Y.; Li, B.; Zhu, E.; Jiang, L.; Zhang, X.; Zhang, S.; Liu, J.; et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv* **2023**, arXiv:2308.08155.
17. Pandya, K.; Holia, M. Automating Customer Service using LangChain: Building custom open-source GPT Chatbot for organizations. *arXiv* **2023**, arXiv:2310.05421.
18. Reimers, N.; Gurevych, I. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In Proceedings of the Empirical Methods in Natural Language Processing (EMNLP), Hong Kong, China, 3–7 November 2019; pp. 3980–3990.
19. Sun, C.; Lo, D.; Khoo, X. Towards more accurate retrieval of duplicate bug reports. In Proceedings of the ACM International Conference on Systems and Software Engineering (ASE), Lawrence, KS, USA, 6–12 November 2011; pp. 253–262.
20. Alipour, A.; Hindle, A.; Stroulia, E. A contextual approach towards more accurate duplicate bug report detection and ranking. *Empir. Softw. Eng.* **2015**, *21*, 565–604. [CrossRef]
21. He, Z.; Marcus, A.; Poshyvanyk, D. Using information retrieval and NLP to classify bug reports. In Proceedings of the International Conference on Program Comprehension (ICPC), Braga, Portugal, 30 June–2 July 2010; pp. 148–157.
22. Cubranic, D.; Murphy, G.C. Automatic bug triage using text categorization. In Proceedings of the Sixteenth International Conference on Software Engineering & Knowledge Engineering (SEKE), Banff, AB, Canada, 20–24 June 2004; pp. 92–97.
23. Rastkar, S.; Murphy, G.C.; Murray, G. Summarizing software artifacts: A case study of bug reports. In Proceedings of the 32nd International Conference on Software Engineering (ICSE), Cape Town, South Africa, 2–8 May 2010; Volume 1, pp. 505–514.
24. Mani, S.; Catherine, R.; Sinha, V.S.; Dubey, A. AUSUM: Approach for unsupervised bug report summarization. In Proceedings of the ACM SIGSOFT 20th International Symposium on the Foundations of Software Engineering (FSE), Cary, NC, USA, 10–17 November 2012; pp. 1–11.
25. Rastkar, S.; Murphy, G.C.; Murray, G. Automatic summarization of bug reports. *IEEE Trans. Softw. Eng.* **2014**, *40*, 366–380. [CrossRef]
26. Lotufo, R.; Malik, Z.; Czarnecki, K. Modeling the 'hurried' bug report reading process to summarize bug reports. *Empir. Softw. Eng.* **2015**, *20*, 516–548. [CrossRef]
27. GitHub. Bug Report Summarization Benchmark Dataset. Available online: https://github.com/GindeLab/Ease_2025_AI_model (accessed on 21 September 2025).
28. Beijing Academy of Artificial Intelligence (BAAI). *BAAI BAAI/bge-large-en-v1.5 (English) Embedding Model*; Technical Report (Model Release); Beijing Academy of Artificial Intelligence (BAAI): Beijing, China, 2023. Available online: https://huggingface.co/BAAI/bge-large-en-v1.5 (accessed on 21 September 2025).
29. Team, Q. Qwen2.5 technical report. *arXiv* **2024**, arXiv:2409.12121. [CrossRef]
30. Team, U. Unsloth: Efficient Fine-Tuning Framework for LLMs. GitHub Repository, 2025. Available online: https://github.com/unslothai/unsloth (accessed on 21 September 2025).
31. Johnson, J.; Douze, M.; Jégou, H. Billion-scale similarity search with GPUs. *IEEE Trans. Big Data* **2021**, *7*, 535–547. [CrossRef]
32. Fang, S.; Tan, Y.-S.; Zhang, T.; Xu, Z.; Liu, H. Effective prediction of bug-fixing priority via weighted graph convolutional networks. *IEEE Trans. Reliab.* **2019**, *7*, 535–547. [CrossRef]
33. Fang, S.; Zhang, T.; Tan, Y.; Jiang, H.; Xia, X.; Sun, X. RepresentThemAll: A universal learning representation of bug reports. In Proceedings of the IEEE/ACM 45th International Conference on Software Engineering (ICSE), Melbourne, Australia, 14–20 May 2023; pp. 602–614.
34. Liu, H.; Yu, Y.; Li, S.; Guo, Y.; Wang, D.; Mao, X. BugSum: Deep context understanding for bug report summarization. In Proceedings of the IEEE/ACM International Conference on Program Comprehension (ICPC), Seoul, Republic of Korea, 13–15 July 2020; pp. 94–105.
35. Shao, Y.; Xiang, B. Towards effective bug report summarization by domain-specific representation learning. *IEEE Access* **2024**, *12*, 37653–37662. [CrossRef]
36. Lamkanfi, A.; Demeyer, S.; Giger, E.; Goethals, B. Predicting the severity of a reported bug. In Proceedings of the IEEE International Conference on Mining Software Repositories (MSR), Cape Town, South Africa, 2–3 May 2010; pp. 1–10. [CrossRef]

37. Sarkar, A.; Rigby, P.C.; Bartalos, B. Improving Bug Triaging with High Confidence Predictions at Ericsson. In Proceedings of the IEEE International Conference on Software Maintenance and Evolution (ICSME), Cleveland, OH, USA, 30 September–4 October 2019; pp. 81–91.
38. Chen, M. Evaluating large language models trained on code. *arXiv* **2021**, arXiv:2107.03374. [CrossRef]
39. Allal, L.B.; Muennighoff, N.; Umapathi, L.K.; Lipkin, B.; von Werra, L. StarCoder: Open source code LLMs. *arXiv* **2023**, arXiv:2305.06161.
40. Rozière, B.; Gehring, J.; Gloeckle, F.; Sootla, S.; Gat, I.; Tan, X.E.; Adi, Y.; Liu, J.; Sauvestre, R.; Remez, T.; et al. Code LLaMA: Open foundation models for code. *arXiv* **2023**, arXiv:2308.12950.
41. Luo, Z.; Xu, C.; Zhao, P.; Sun, Q.; Geng, X.; Hu, W.; Tao, C.; Ma, J.; Lin, Q.; Jiang, D. WizardCoder: Empowering code LLMs to speak code fluently. *arXiv* **2023**, arXiv:2306.08568.
42. Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegreffe, S.; Alon, U.; Dziri, N.; Prabhumoye, S.; Yang, Y.; et al. Self-Refine: Iterative refinement with feedback. *arXiv* **2023**, arXiv:2303.17651.
43. Chen, X.; Lin, M.; Schärli, N.; Zhou, D. Teaching LLMs to self-debug. *arXiv* **2023**, arXiv:2304.05125.
44. Gou, Z.; Shao, Z.; Gong, Y.; Shen, Y.; Yang, Y.; Duan, N.; Chen, W. CRITIC: LLMs can self-correct with tool-interactive critiquing. *arXiv* **2023**, arXiv:2305.11738.
45. Nye, M.; Andreassen, A.J.; Gur-Ari, G.; Michalewski, H.; Austin, J.; Bieber, D.; Dohan, D.; Lewkowycz, A.; Bosma, M.; Luan, D.; et al. Show your work: Scratchpads for intermediate reasoning. *arXiv* **2021**, arXiv:2112.00114.
46. Papineni, K.; Roukos, S.; Ward, T.; Zhu, W.-J. BLEU: A method for automatic evaluation of machine translation. In Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics, Philadelphia, PA, USA, 6–7 July 2002; pp. 311–318.
47. Zhang, T.; Kishore, V.; Wu, F.; Weinberger, K.Q.; Artzi, Y. BERTScore: Evaluating text generation with BERT. In Proceedings of the 8th International Conference on Learning Representations (ICLR 2020), Addis Ababa, Ethiopia, 26–30 April 2020.
48. Fabbri, A.; Kryscinski, V.; McCann, B.; Xiong, C.; Socher, R.; Radev, D. SummEval: Re-evaluating summarization evaluation. *Trans. Assoc. Comput. Linguist.* **2021**, *9*, 391–409. [CrossRef]
49. Eghbali, A.; Pradel, M. CrystalBLEU: Precisely and efficiently measuring code generation quality. In Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, Belfast, UK, 10–12 June 2022; pp. 1–12.
50. Zhou, S.; Alon, U.; Agarwal, S.; Neubig, G. CodeBERTScore: Evaluating code generation with BERT-based similarity. *arXiv* **2023**, arXiv:2302.05527.
51. Fu, J.; Ng, S.-K.; Jiang, Z.; Liu, P. GPTScore: Evaluate as you desire. *arXiv* **2023**, arXiv:2302.04166. [CrossRef]
52. Wang, Z.; Zhou, S.; Fried, D.; Neubig, G. Execution-based evaluation for open-domain code generation. *arXiv* **2022**, arXiv:2212.10481.
53. Lu, S.; Guo, D.; Ren, R.; Huang, J.; Svyatkovskiy, A.; Blanco, A.; Clement, C.; Drain, D.; Jiang, D.; Tang, D.; et al. CodeXGLUE: A machine learning benchmark dataset for code understanding and generation. *arXiv* **2021**, arXiv:2102.04664. [CrossRef]
54. Yasunaga, M.; Liang, P. Break-it-fix-it: Unsupervised learning for program repair. In Proceedings of the 38th International Conference on Machine Learning (ICML), Virtual, 18–24 July 2021; pp. 11941–11952.
55. Ye, H.; Martinez, M.; Durieux, T.; Monperrus, M. A comprehensive study of automated program repair on QuixBugs. *J. Syst. Softw.* **2021**, *171*, 110825. [CrossRef]
56. GitHub. AgentReport: A Multi-Agent LLM Approach for Automated and Reproducible Bug Report Generation. Available online: https://github.com/insui12/AgentReport (accessed on 1 November 2025).